

Section 10.2 - A temperature-forecasting example

December 9, 2025

1 A temperature-forecasting example

- Task: predict the temperature 24 hours in the future given a timeseries of hourly measurements of weather quantities
- Average temperature of a month – easy, but day-scale temperatures is challenging!
- Objective: highlight the fundamental difference between timeseries data with others
- Recurrent neural networks (RNNs) shine on this type of problems

1.1 The Jena weather dataset

- Dataset measured at weather station at Max Planck Institute for Biogeochemistry in Jena, Germany
- 14 weather-related quantities: recorded every 10 minutes during 2009-2016
- Create the temperature array as the target of prediction
- Discard the Data-time column, the rest as features

```
[1]: import tensorflow as tf

# ---- Compatibility patch for older TF + newer Keras RNN backend ----
def _install_defun_with_attributes(internal_ns):
    # internal_ns should have a .function submodule
    if hasattr(internal_ns, "function"):
        fn_mod = internal_ns.function
        if not hasattr(fn_mod, "defun_with_attributes"):
            def defun_with_attributes(func=None, **kwargs):
                # Ignore extra attributes; just wrap with tf.function
                return tf.function(func, autograph=kwargs.get("autograph",
↪False))
            fn_mod.defun_with_attributes = defun_with_attributes

# Patch tf.__internal__
if hasattr(tf, "__internal__"):
    _install_defun_with_attributes(tf.__internal__)

# Patch tf.compat.v2.__internal__ (this is the one from your error trace)
if hasattr(tf, "compat") and hasattr(tf.compat, "v2") and hasattr(tf.compat.v2,
↪ "__internal__"):
    _install_defun_with_attributes(tf.compat.v2.__internal__)
```

```
# Now import keras from TensorFlow (avoid plain "import keras")
from tensorflow import keras
from tensorflow.keras import layers

print("TF version:", tf.__version__)
print("Keras from tf:", keras.__version__)
```

TF version: 2.13.1

Keras from tf: 2.10.0

```
[2]: import os

fname = os.path.join("jena_climate_2009_2016.csv")

with open(fname) as f:
    data = f.read()

lines = data.split("\n")
header = lines[0].split(",")
lines = lines[1:]
print(header)
print(len(lines))
```

```
['Date Time', 'p (mbar)', 'T (degC)', 'Tpot (K)', 'Tdew (degC)', 'rh (%)', 'VPmax (mbar)', 'VPact (mbar)', 'VPdef (mbar)', 'sh (g/kg)', 'H2OC (mmol/mol)', 'rho (g/m**3)', 'wv (m/s)', 'max. wv (m/s)', 'wd (deg)']
420451
```

1.1.1 Parsing the data

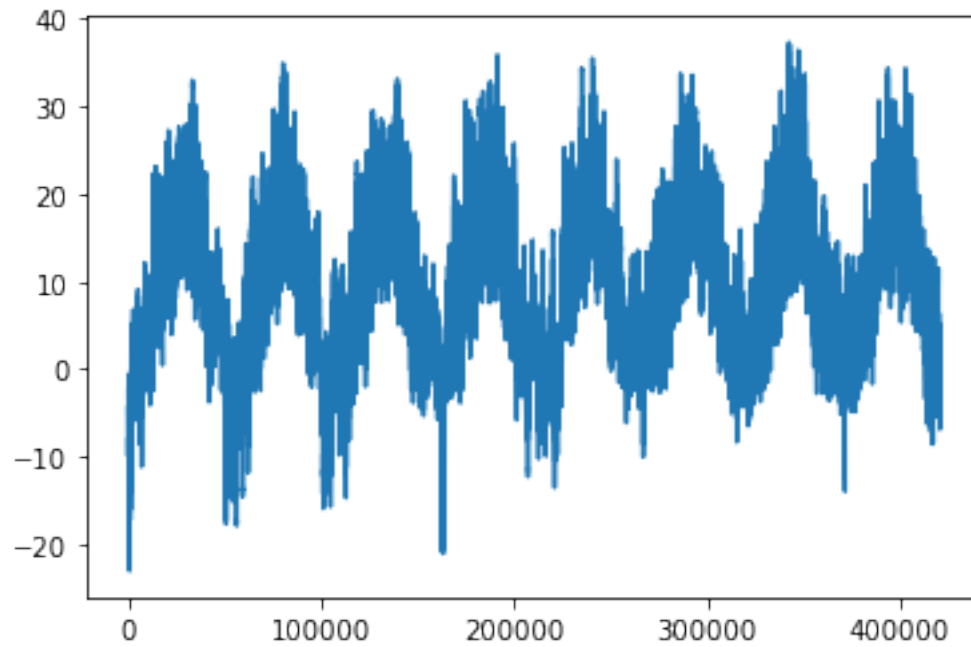
- 420,551 lines of data – each line is a timestep
- Observe yearly periodicity

```
[3]: import numpy as np

temperature = np.zeros((len(lines),))
raw_data = np.zeros((len(lines), len(header) - 1))
for i, line in enumerate(lines):
    values = [float(x) for x in line.split(",")[1:]]
    temperature[i] = values[1]
    raw_data[i, :] = values[2:]

# Plotting the temperature timeseries
from matplotlib import pyplot as plt
plt.plot(range(len(temperature)), temperature)
```

```
[3]: [<matplotlib.lines.Line2D at 0x7f7071702af0>]
```

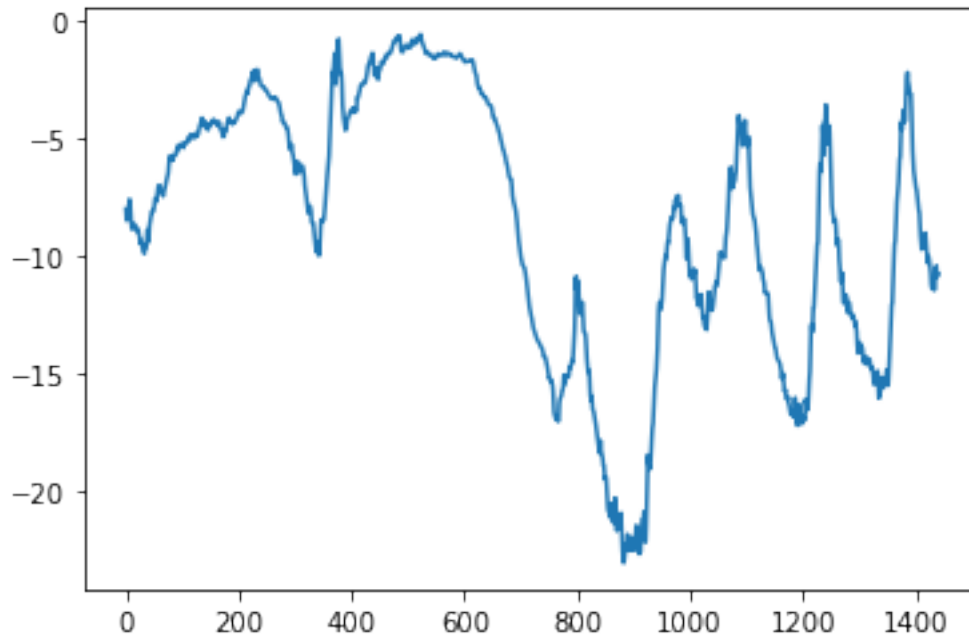


1.1.2 The first 10 days of the temperature timeseries

- Observe daily periodicity
- $24 \times 6 = 144$ data points per day

```
[4]: plt.plot(range(1440), temperature[:1440])
```

```
[4]: [<matplotlib.lines.Line2D at 0x7f707169e7f0>]
```



1.1.3 Data splitting

- First 50% for training, next 25% for validation and last 25% for testing
- Timeseries data: more recent data for validation and testing than for training - predict future, given the past, not the reverse!

```
[5]: num_train_samples = int(0.5 * len(raw_data))
num_val_samples = int(0.25 * len(raw_data))
num_test_samples = len(raw_data) - num_train_samples - num_val_samples
print("num_train_samples:", num_train_samples)
print("num_val_samples:", num_val_samples)
print("num_test_samples:", num_test_samples)
```

```
num_train_samples: 210225
num_val_samples: 105112
num_test_samples: 105114
```

1.1.4 Preparing the data

- Each timeseries on a different scale -> Normalization each independently
- Mean and standard deviation are computed from the training dataset

```
[6]: mean = raw_data[:num_train_samples].mean(axis=0)
raw_data -= mean
std = raw_data[:num_train_samples].std(axis=0)
raw_data /= std
```

1.1.5 Instantiating datasets for training, validation, and testing

- What does a sample look like?
 - Samples in dataset are highly redundant -> Generate the samples on the fly
 - `timeseries_dataset_from_array()` - given an array of timeseries data, provide windows extracted from the original timeseries
- 1) Sample rate = 6: keep one data point per hour
 - 2) Sequence length = 120: back to 5 days
 - 3) Delay = sample rate * (sequence length + 24 - 1)
 - 4) A dataset – a streaming generator, not a concrete tensor

```
[7]: import numpy as np
from tensorflow import keras

sampling_rate = 6
sequence_length = 120
delay = sampling_rate * (sequence_length + 24 - 1)
batch_size = 256

train_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[::-delay],
    targets=temperature[delay:],
    sampling_rate=sampling_rate,
    sequence_length=sequence_length,
    shuffle=True,
    batch_size=batch_size,
    start_index=0,
    end_index=num_train_samples)

val_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[::-delay],
    targets=temperature[delay:],
    sampling_rate=sampling_rate,
    sequence_length=sequence_length,
    shuffle=True,
    batch_size=batch_size,
    start_index=num_train_samples,
    end_index=num_train_samples + num_val_samples)

test_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[::-delay],
    targets=temperature[delay:],
    sampling_rate=sampling_rate,
    sequence_length=sequence_length,
    shuffle=True,
    batch_size=batch_size,
```

```
start_index=num_train_samples + num_val_samples)
```

1.1.6 Inspecting the datasets

```
[8]: for samples, targets in train_dataset:
      print("samples shape:", samples.shape)
      print("targets shape:", targets.shape)
      break
```

samples shape: (256, 120, 14)

targets shape: (256,)

1.2 A non-machine-learning baseline

- Sanity check: baseline to beat to demonstrate the usefulness of more-advanced DL models
- Make sure any further complexity is legitimate and delivers real benefits
- common baseline = same as last day (by continuity of temperature)
- Unnormalization – multiply the standard deviation and add the mean
- validation MAE: 2.44 & test MAE: 2.62 degrees Celsius

```
[9]: def evaluate_naive_method(dataset):
      total_abs_err = 0.
      samples_seen = 0
      for samples, targets in dataset:
          preds = samples[:, -1, 1] * std[1] + mean[1]
          total_abs_err += np.sum(np.abs(preds - targets))
          samples_seen += samples.shape[0]
      return total_abs_err / samples_seen

print(f"Validation MAE: {evaluate_naive_method(val_dataset):.2f}")
print(f"Test MAE: {evaluate_naive_method(test_dataset):.2f}")
```

Validation MAE: 2.44

Test MAE: 2.62

1.3 A densely connected trial

- Model with two Dense layers
- MSE loss & MAE as a metric
- Regression: no activation at the last Dense layer
- Fail to find a simple solution to a simple problem

```
[10]: # Hide the warning messages
import tensorflow as tf
import logging

tf.get_logger().setLevel(logging.ERROR)

from tensorflow import keras
```

```

from tensorflow.keras import layers

inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.Flatten()(inputs)
x = layers.Dense(16, activation="relu")(x)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("jena_dense.h5",
                                    save_best_only=True)
]
model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history = model.fit(train_dataset,
                    epochs=10,
                    validation_data=val_dataset,
                    callbacks=callbacks)

model = keras.models.load_model("jena_dense.h5")
print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")

```

```

Epoch 1/10
819/819 [=====] - 6s 7ms/step - loss: 12.6590 - mae:
2.7428 - val_loss: 11.5341 - val_mae: 2.6923
Epoch 2/10
819/819 [=====] - 6s 7ms/step - loss: 8.8559 - mae:
2.3421 - val_loss: 10.3371 - val_mae: 2.5244
Epoch 3/10
819/819 [=====] - 6s 7ms/step - loss: 8.1946 - mae:
2.2526 - val_loss: 9.9502 - val_mae: 2.4783
Epoch 4/10
819/819 [=====] - 6s 8ms/step - loss: 7.7522 - mae:
2.1904 - val_loss: 11.3946 - val_mae: 2.6792
Epoch 5/10
819/819 [=====] - 6s 8ms/step - loss: 7.4124 - mae:
2.1407 - val_loss: 10.9947 - val_mae: 2.6206
Epoch 6/10
819/819 [=====] - 7s 8ms/step - loss: 7.1770 - mae:
2.1068 - val_loss: 10.9127 - val_mae: 2.6204
Epoch 7/10
819/819 [=====] - 7s 8ms/step - loss: 6.9638 - mae:
2.0753 - val_loss: 11.2410 - val_mae: 2.6605
Epoch 8/10
819/819 [=====] - 7s 8ms/step - loss: 6.8142 - mae:
2.0529 - val_loss: 11.2412 - val_mae: 2.6591
Epoch 9/10
819/819 [=====] - 7s 8ms/step - loss: 6.6831 - mae:
2.0338 - val_loss: 12.5904 - val_mae: 2.8033

```

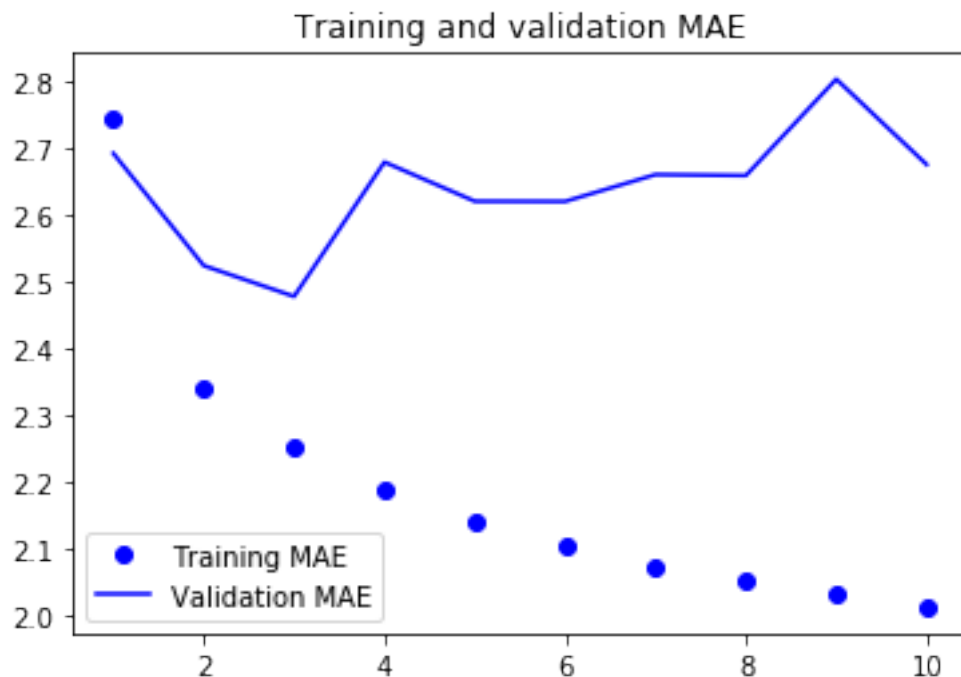
Epoch 10/10
819/819 [=====] - 7s 8ms/step - loss: 6.5609 - mae:
2.0138 - val_loss: 11.5158 - val_mae: 2.6756
405/405 [=====] - 2s 5ms/step - loss: 10.7797 - mae:
2.5907
Test MAE: 2.59

1.3.1 Plotting the training results

- Fail to beat the non-machine-learning baseline
- Dense layers flatten the timeseries data -> remove notion of time from the data

```
[11]: import matplotlib.pyplot as plt

loss = history.history["mae"]
val_loss = history.history["val_mae"]
epochs = range(1, len(loss) + 1)
plt.figure()
plt.plot(epochs, loss, "bo", label="Training MAE")
plt.plot(epochs, val_loss, "b", label="Validation MAE")
plt.title("Training and validation MAE")
plt.legend()
plt.show()
```



1.4 A 1D convolutional trial

- A temporal convnet reuses same representations across different timesteps like a spatial convnet reuses the same representations across different locations in an image
- What is SeparableConv1D?

```
[12]: inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.Conv1D(8, 24, activation="relu")(inputs)
x = layers.MaxPooling1D(2)(x)
x = layers.Conv1D(8, 12, activation="relu")(x)
x = layers.MaxPooling1D(2)(x)
x = layers.Conv1D(8, 6, activation="relu")(x)
x = layers.GlobalAveragePooling1D()(x)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("jena_conv.h5",
                                    save_best_only=True)
]

model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])

history = model.fit(train_dataset,
                    epochs=10,
                    validation_data=val_dataset,
                    callbacks=callbacks)

model = keras.models.load_model("jena_conv.h5")

print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")
```

```
Epoch 1/10
819/819 [=====] - 8s 8ms/step - loss: 19.9672 - mae:
3.5136 - val_loss: 15.9737 - val_mae: 3.1316
Epoch 2/10
819/819 [=====] - 6s 8ms/step - loss: 14.7317 - mae:
3.0512 - val_loss: 16.4029 - val_mae: 3.2661
Epoch 3/10
819/819 [=====] - 7s 8ms/step - loss: 13.6446 - mae:
2.9403 - val_loss: 26.7955 - val_mae: 4.1759
Epoch 4/10
819/819 [=====] - 7s 9ms/step - loss: 12.8110 - mae:
2.8456 - val_loss: 16.1042 - val_mae: 3.1878
Epoch 5/10
819/819 [=====] - 7s 9ms/step - loss: 12.2611 - mae:
2.7820 - val_loss: 14.0787 - val_mae: 2.9643
Epoch 6/10
819/819 [=====] - 7s 9ms/step - loss: 11.8009 - mae:
```

```

2.7280 - val_loss: 13.9102 - val_mae: 2.9317
Epoch 7/10
819/819 [=====] - 7s 9ms/step - loss: 11.4202 - mae:
2.6801 - val_loss: 14.9227 - val_mae: 3.0328
Epoch 8/10
819/819 [=====] - 7s 9ms/step - loss: 11.1491 - mae:
2.6469 - val_loss: 14.4553 - val_mae: 2.9804
Epoch 9/10
819/819 [=====] - 7s 8ms/step - loss: 10.8580 - mae:
2.6095 - val_loss: 14.8515 - val_mae: 3.0314
Epoch 10/10
819/819 [=====] - 7s 9ms/step - loss: 10.6331 - mae:
2.5826 - val_loss: 14.1441 - val_mae: 2.9512
405/405 [=====] - 2s 5ms/step - loss: 15.8930 - mae:
3.1556
Test MAE: 3.16

```

1.4.1 Plotting the training results

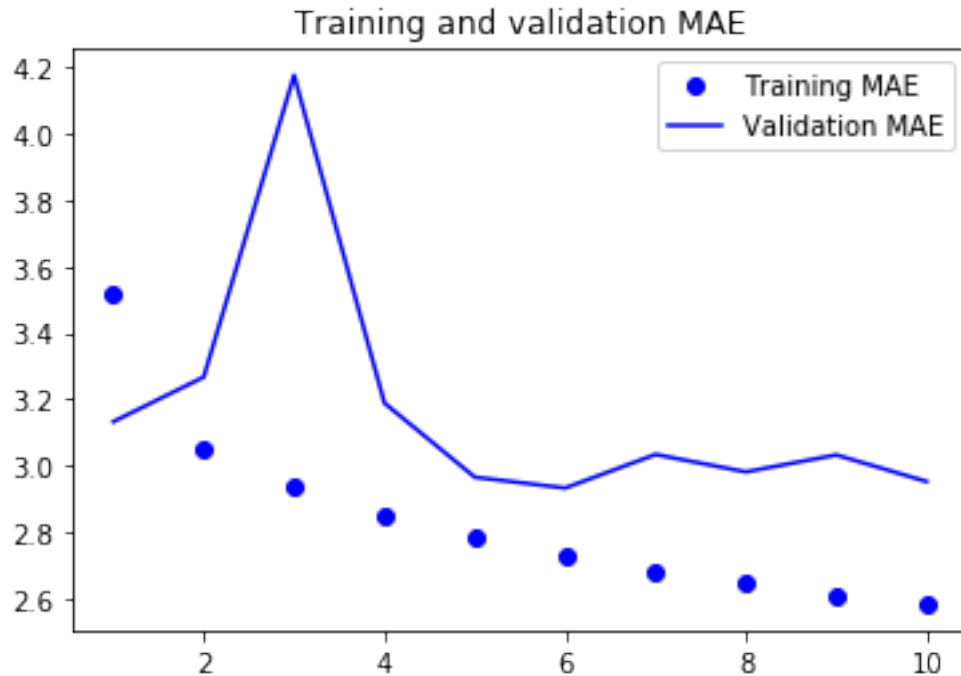
- Even worse! Again, fail to beat the non-machine-learning baseline
- Convnets treat every segment of data in the same way
- Pooling destroys order information

```

[13]: import matplotlib.pyplot as plt

loss = history.history["mae"]
val_loss = history.history["val_mae"]
epochs = range(1, len(loss) + 1)
plt.figure()
plt.plot(epochs, loss, "bo", label="Training MAE")
plt.plot(epochs, val_loss, "b", label="Validation MAE")
plt.title("Training and validation MAE")
plt.legend()
plt.show()

```



1.5 A first recurrent baseline - A simple LSTM-based model

- Leveraging feature engineering & relevant architecture priors are essential
- Timeseries data (as sequences) - causality and order matter
- Understand RNNs and LSTM layers later!

```
[14]: import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Model definition
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.LSTM(16)(inputs)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

# Callbacks
callbacks = [
    keras.callbacks.ModelCheckpoint(
        "jena_lstm.h5",
        save_best_only=True
    )
]

# Compile
```

```

model.compile(
    optimizer="rmsprop",
    loss="mse",
    metrics=["mae"]
)

# Train
history = model.fit(
    train_dataset,
    epochs=10,
    validation_data=val_dataset,
    callbacks=callbacks
)

# Reload best model and evaluate
model = keras.models.load_model("jena_lstm.h5")
test_loss, test_mae = model.evaluate(test_dataset)
print(f"Test MAE: {test_mae:.2f}")

```

```

Epoch 1/10
819/819 [=====] - 75s 90ms/step - loss: 45.5080 - mae:
4.9401 - val_loss: 13.1267 - val_mae: 2.7470
Epoch 2/10
819/819 [=====] - 73s 89ms/step - loss: 11.4203 - mae:
2.6115 - val_loss: 9.5334 - val_mae: 2.4178
Epoch 3/10
819/819 [=====] - 73s 89ms/step - loss: 9.6763 - mae:
2.4229 - val_loss: 9.1983 - val_mae: 2.3749
Epoch 4/10
819/819 [=====] - 73s 89ms/step - loss: 9.0667 - mae:
2.3485 - val_loss: 9.4523 - val_mae: 2.4054
Epoch 5/10
819/819 [=====] - 73s 89ms/step - loss: 8.7154 - mae:
2.3046 - val_loss: 9.5636 - val_mae: 2.4192
Epoch 6/10
819/819 [=====] - 72s 88ms/step - loss: 8.4125 - mae:
2.2653 - val_loss: 9.7379 - val_mae: 2.4327
Epoch 7/10
819/819 [=====] - 72s 88ms/step - loss: 8.2013 - mae:
2.2390 - val_loss: 9.7447 - val_mae: 2.4366
Epoch 8/10
819/819 [=====] - 72s 88ms/step - loss: 8.0203 - mae:
2.2132 - val_loss: 9.8722 - val_mae: 2.4589
Epoch 9/10
819/819 [=====] - 72s 88ms/step - loss: 7.8651 - mae:
2.1923 - val_loss: 9.7374 - val_mae: 2.4343
Epoch 10/10
819/819 [=====] - 72s 88ms/step - loss: 7.7436 - mae:

```

```
2.1757 - val_loss: 9.7875 - val_mae: 2.4435
405/405 [=====] - 7s 16ms/step - loss: 10.1448 - mae:
2.4964
Test MAE: 2.50
```

1.5.1 Plotting the training results

Better! validation MAE 2.36 degrees & test MAE 2.55

```
[15]: import matplotlib.pyplot as plt

loss = history.history["mae"]
val_loss = history.history["val_mae"]
epochs = range(1, len(loss) + 1)
plt.figure()
plt.plot(epochs, loss, "bo", label="Training MAE")
plt.plot(epochs, val_loss, "b", label="Validation MAE")
plt.title("Training and validation MAE")
plt.legend()
plt.show()
```

